الگوریتم Multilevel Queue Scheduling
گزارش جامع تمرین اول: معرفی کامل

زمان‌بندی چندسطحی صف‌ها در سیستم‌عامل — تحلیل، پیاده‌سازی و ارزیابی آزمایشی

عنوان تمرینمعرفی کامل الگوریتم Multilevel Queue Scheduling
درسسیستم‌عامل پیشرفته
موضوعزمان‌بندی پردازنده و الگوریتم‌های صف چندسطحی
قالب تحویلگزارش فارسی (Word و PDF) + کد اجرایی پایتون + نمودارها
مخزن پروژهgithub.com/kaspian021/Multilevel-Queue-Scheduling
شاخه تمرین Exercise-1-Algorithm-Introduction
تعداد صفحاتمطابق خروجی واقعی تولیدشده (بیش از ۲۰ صفحه)
وضعیتنسخه کامل — اعداد از اجرای واقعی کد استخراج شده‌اند

فهرست مطالب

چکیده، کلیدواژه‌ها و نقشه گزارش	3
مقدمه و بیان مسئله	4
اهداف، پرسش‌ها و روش مطالعه	5
مبانی نظری (۱): پردازنده، پردازه و نخ	6
مبانی نظری (۲): چرخه اجرا و انواع Burst	7
مبانی نظری (۳): معیارهای ارزیابی	8
مبانی نظری (۴): صف آماده، تعویض زمینه و بهره‌وری	9
معرفی MLQ: فلسفه و اهداف طراحی	10
ساختار صف‌ها و نگاشت به کلاس‌های کاری	11
تخصیص پردازه‌ها به صف‌ها	12
سیاست زمان‌بندی داخل هر صف	13
مدیریت اولویت بین صف‌ها	14
فلوچارت الگوریتم	15
شبه‌کد (بخش ۱): حلقه اصلی زمان‌بند	16
شبه‌کد (بخش ۲): اجرا، پیش‌گیری و پیری	17
مثال عددی حل‌شده (۱): صورت مسئله	18
مثال عددی حل‌شده (۲): اجرای گام‌به‌گام	19
مثال عددی حل‌شده (۳): نمودار گانت	20
مثال عددی حل‌شده (۴): جدول محاسبات و نتایج	21
سناریوی دوم: مقایسه سیاست‌های درون‌صفی	22
تحلیل گرسنگی در صف چندسطحی	23
روش‌های جلوگیری از گرسنگی	24
تحلیل پیچیدگی زمانی و فضایی	25
پیاده‌سازی پایتون (۱): ساختار و چرخه استفاده	26
پیاده‌سازی پایتون (۲): ورودی، خروجی و مدیریت خطا	27
سناریوها و نتایج آزمایش‌ها	28
تحلیل نتایج	29
سربار زمان‌بندی و هزینه تصمیم‌گیری	30
مزایا، معایب و مقایسه با سیاست‌های تک‌صفی	31
محدودیت‌ها، فرضیات و کاربردهای واقعی	32
نتیجه‌گیری و پیشنهادها	33
منابع	34
پیوست: فهرست خروجی‌ها و وضعیت فایل‌ها	35

چکیده، کلیدواژه‌ها و نقشه گزارش

چکیده

یک سیاست بین‌صفی (اولویت ثابت یا برش زمانی) مشخص می‌کند کدام صف در هر لحظه از پردازنده استفاده کند. را به چند صف با اولویت متفاوت تقسیم می‌کند، برای هر صف یک سیاست زمان‌بندی مستقل تعیین می‌نماید و با چندسطحی (Multilevel Queue یا MLQ) به‌طور کامل معرفی می‌شود؛ الگوریتمی که مجموعه پردازه‌های آماده و کیفیت آن مستقیماً بر پاسخ‌دهی، کارایی و عدالت تخصیص منابع اثر می‌گذارد. در این گزارش، الگوریتم زمان‌بندی زمان‌بندی پردازنده یکی از بنیادی‌ترین وظایف سیستم‌عامل است

می‌شوند؛ در سناریوی گرسنگی این گزارش، زمان انتظار پردازه کم‌اولویت از ۲۰ واحد زمان به ۶ واحد کاهش یافت. دقیقی بر اولویت‌بندی می‌دهد، اما در صورت نبود سازوکار پیری (Aging)، پردازه‌های صف‌های پایین دچار گرسنگی بهره‌وری پردازنده و تعداد تعویض زمینه از اجرای واقعی برنامه استخراج شده‌اند. نتایج نشان می‌دهد که MLQ کنترل می‌کند. چهار سناریوی آزمایشی طراحی و اجرا شده و همه شاخص‌های زمان انتظار، زمان چرخش، زمان پاسخ، که چهار سیاست درون‌صفی FCFS، RR، SJF و SRTF و دو سیاست بین‌صفی اولویت ثابت و برش زمانی را پشتیبانی در این تمرین، افزون‌بر تبیین نظری، یک شبیه‌ساز گسسته و قابل‌اجرا به زبان پایتون توسعه داده شده

کلیدواژه‌ها

اولویت ثابت؛ برش زمانی؛ زمان انتظار؛ زمان چرخش؛ زمان پاسخ؛ تعویض زمینه؛ گرسنگی؛ پیری؛ نمودار گانت؛ پایتون. زمان‌بندی پردازنده؛ صف چندسطحی؛

نقشه گزارش

- بخش ۱ تا ۳: مبانی نظری، تعاریف تخصصی و معیارهای ارزیابی.
- بخش ۴ تا ۷: معرفی MLQ، ساختار صف‌ها، سیاست‌های درون‌صفی و بین‌صفی.
- بخش ۸ تا ۱۰: فلوچارت، شبه‌کد و مثال عددی کامل حل‌شده.
- بخش ۱۱ تا ۱۴: سناریوهای آزمایشی، تحلیل نتایج، گرسنگی و پیری.
- بخش ۱۵ تا ۱۹: پیچیدگی، پیاده‌سازی، مزایا و معایب، محدودیت‌ها و کاربردها.

مقدمه و بیان مسئله

بیان مسئله

پردازنده استفاده کند. این تصمیم توسط «زمان‌بند» (Scheduler) و بر پایه یک «الگوریتم زمان‌بندی» گرفته می‌شود. آماده معمولاً بیشتر از تعداد هسته‌های پردازنده است. سیستم‌عامل باید در هر لحظه تصمیم بگیرد کدام پردازه از در یک سیستم اشتراک‌زمانی، تعداد پردازه‌های

یک سیاست یکسان برای همه این گروه‌ها نمی‌تواند هم‌زمان پاسخ‌دهی خوب، عدالت و کارایی بالا فراهم کند. نیستند: برخی تعاملی و حساس به تأخیرند، برخی پردازش‌محور و بلندمدت، و برخی متعلق به سیستم و بحرانی. مسئله اصلی این است که پردازه‌ها از نظر ماهیت یکسان

چرا MLQ مطرح شد؟

- الگوریتم FCFS ساده است اما میانگین زمان انتظار بالایی دارد و برای کار تعاملی مناسب نیست.
- SJF میانگین زمان انتظار را بهینه می‌کند اما نیازمند دانستن طول Burst است و می‌تواند باعث گرسنگی شود. الگوریتم
- RR پاسخ‌دهی خوبی دارد اما سربار تعویض زمینه را بالا می‌برد و میانگین زمان چرخش را افزایش می‌دهد. الگوریتم
- الگوریتم Priority انعطاف‌پذیر است اما نیازمند تعریف اولویت و مدیریت گرسنگی است.
- .با ترکیب این سیاست‌ها در لایه‌های مختلف، امکان رفتار متفاوت با هر گروه را فراهم می‌کند MLQ

صورت‌بندی رسمی مسئله

تخصیص دهد، به‌گونه‌ای که ساختار اولویت‌ها رعایت شود و شاخص‌های ارزیابی در محدوده قابل‌قبول باقی بمانند. یافتن یک ترتیب اجراست که بر پایه سیاست تعیین‌شده، پردازنده را در هر تیک زمانی به یکی از پردازه‌های آماده مجموعه‌ای از n پردازه با زمان ورود، طول Burst و سطح page مشخص داده شده است. هدف

:محدوده اعتبار مدلیک شبیه‌ساز آموزشی و قابل‌سنجش است و جایگزین زمان‌بند واقعی هسته سیستم‌عامل نیست. در این گزارش «تیک زمانی» واحد گسسته شبیه‌سازی است. مدل ارائه‌شده

اهداف، پرسش‌ها و روش مطالعه

هدف کلی

از الگوریتم MLQ شامل مبانی نظری، طراحی، شبه‌کد، پیاده‌سازی اجرایی و ارزیابی کمی بر پایه سناریوهای آزمایشی. ارائه یک معرفی کامل، دقیق و قابل‌اتکا

اهداف جزئی

- تبیین دقیق مفاهیم پایه: پردازنده، پردازه، نخ، Burst و تعویض زمینه.
- تعریف عملیاتی و ریاضی شاخص‌های Turnaround، Waiting و Response Time.
- تشریح ساختار صف‌ها، نحوه تخصیص پردازه به صف و سیاست‌های درون‌صفی.
- مقایسه دو سیاست مدیریت اولویت بین صف‌ها: اولویت ثابت و برش زمانی.
- ارائه فلوچارت و شبه‌کد کامل و قابل‌تبدیل به کد.
- حل کامل یک مثال عددی با نمودار گانت و جدول محاسبات.
- پیاده‌سازی پایتون با مدیریت خطا، تست خودکار و خروجی قابل‌بررسی.
- تحلیل گرسنگی و ارزیابی اثر سازوکار پیری.

پرسش‌های پژوهش

- آیا ساختار چندسطحی می‌تواند زمان پاسخ کارهای تعاملی را بدون کاهش بهره‌وری بهبود دهد؟
- کدام سیاست درون‌صفی برای یک بار کاری مشخص، میانگین زمان انتظار کمتری می‌دهد؟
- شدت گرسنگی در صف‌های پایین چقدر است و پیری تا چه اندازه آن را مهار می‌کند؟
- هزینه سربار زمان‌بندی در قالب تعویض زمینه چقدر است؟

روش مطالعه

- مطالعه کتابخانه‌ای برای تدوین مبانی نظری و تعاریف.
- طراحی مدل گسسته و پیاده‌سازی آن در پایتون.
- طراحی چهار سناریوی آزمایشی و اجرای خودکار آن‌ها.
- تولید نمودارها از خروجی واقعی برنامه و تحلیل عددی نتایج.
- تطبیق نتایج کد با محاسبات دستی در یک مثال مرجع.

مبانی نظری (۱): پردازنده، پردازه و نخ

پردازنده و زمان‌بند

انتخاب پردازه بعدی از میان پردازه‌های آماده را بر عهده دارد و «dispatcher» عمل تحویل پردازنده را انجام می‌دهد. منبعی است که در هر لحظه تنها در اختیار یک پردازه قرار می‌گیرد. «زمان‌بند کوتاه‌مدت» یا CPU Scheduler وظیفه پردازنده

پردازه

- پردازه یک برنامه در حال اجراست و شامل فضای آدرس، ثبات‌ها، فایل‌های باز و اطلاعات زمان‌بندی است.
- هر پردازه در یک چرخه عمر بین حالت‌های New، Ready، Running، Waiting و Terminated جابه‌جا می‌شود.
- بلوک کنترل پردازه (PCB) اطلاعات لازم برای توقف و ادامه اجرا را نگه می‌دارد.
- زمان‌بند تنها بر پردازه‌هایی تصمیم می‌گیرد که در حالت Ready هستند.

نخ

- نخ کوچک‌ترین واحد اجرایی است و بخشی از یک پردازه محسوب می‌شود.
- نخ‌های یک پردازه حافظه و منابع را به اشتراک می‌گذارند اما هر کدام زمینه اجرایی مستقل دارند.
- مدرن، واحد زمان‌بندی معمولاً نخ است؛ در این گزارش برای سادگی از واژه پردازه استفاده می‌شود. در سیستم‌عامل‌های
- در مدل این تمرین، هر ردیف ورودی یک واحد زمان‌بندی مستقل است.

تفاوت پردازه و نخ در یک نگاه

نخ	پردازه	ویژگی
اشتراکی با نخ‌های همان پردازه	مستقل	فضای آدرس
پایین	بالا	هزینه ایجاد
پایین‌تر	بالا	هزینه تعویض زمینه
پایین	بالا	استقلال در برابر خطا
معمولاً بله	معمولاً نه	واحد زمان‌بندی در سیستم‌های مدرن

جدول 1. مقایسه پردازه و نخ از منظر زمان‌بندی

مبانی نظری (۲): چرخه اجرا و انواع Burst

چرخه CPU و I/O

(I/O Burst). هر پردازه با یک CPU Burst شروع می‌شود و پس از آن در صورت نیاز وارد فاز I/O می‌گردد.
بین دو فاز انجام می‌شود: یک فاز استفاده از پردازنده (CPU Burst) و یک فاز انتظار برای عملیات ورودی/خروجی
اجرای هر پردازه به‌صورت متناوب

تعریف Burst Time

Burst ،یا زمان رگبار، مدت‌زمانی است که پردازه برای اجرا به پردازنده نیاز دارد. در مدل این تمرین Burst Time
.یک عدد صحیح بر حسب تیک است و در زمان ورود معلوم فرض می‌شود

نکات مدل‌سازی Burst

- در سیستم واقعی طول Burst از پیش معلوم نیست و تنها با پیش‌بینی تخمین زده می‌شود.
- الگوریتم SJF در عمل به‌صورت تقریبی و با میانگین نمایی اجرا می‌شود.
- در این شبیه‌ساز، دانستن Burst یک فرض صریح مدل است و در بخش محدودیت‌ها ذکر شده است.
- توزیع Burst در بار کاری واقعی معمولاً دارای دنباله سنگین است؛ یعنی تعداد کمی پردازه بسیار طولانی‌اند.

انواع زمان‌بند بر پایه تصمیم‌گیری

مثال	زمان تصمیم	نوع زمان‌بند
کنترل درجه چندبرنامه‌سازی	پذیرش پردازه در سیستم	بلندمدت
Swapping	تعلیق و بازگردانی	میان‌مدت
FCFS، RR، SJF، MLQ	هر تیک یا هر رویداد	کوتاه‌مدت

جدول 2. سطوح تصمیم‌گیری در زمان‌بندی

پیش‌گیرانه در برابر غیرپیش‌گیرانه

- در زمان‌بندی غیرپیش‌گیرانه، پردازنده تا پایان Burst یا ورود به انتظار آزاد نمی‌شود.
- در زمان‌بندی پیش‌گیرانه، می‌توان پردازه در حال اجرا را متوقف و پردازه دیگری را جایگزین کرد.
- در سطح بین‌صفی پیش‌گیرانه است؛ در سطح درون‌صفی بسته به سیاست می‌تواند هر دو باشد MLQ
.در این پیاده‌سازی
- .سیاست‌های غیرپیش‌گیرانه در نظر گرفته می‌شوند SJF و FCFS سیاست‌های پیش‌گیرانه و SRTF و RR

مبانی نظری (۳): معیارهای ارزیابی

تعاریف عملیاتی

تعریف	نماد
لحظه ورود پردازه به صف آماده	Arrival Time یا AT
مقدار زمان پردازنده موردنیاز پردازه	Burst Time یا BT
لحظه پایان اجرای پردازه	Finish Time یا CT
؛ کل زمان حضور پردازه در سیستمCT - AT	TAT (Turnaround Time)
؛ زمان صرف‌شده در صف آمادهTAT - BT	WT (Waiting Time)
نخستین لحظه اجرا - AT	RT (Response Time)
تغییر پردازه در حال اجرا روی پردازنده	Context Switch

جدول 3. نمادها و تعاریف مورد استفاده در این گزارش

روابط ریاضی

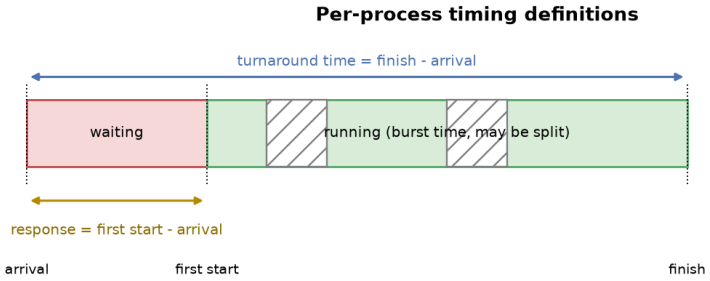
$$TAT_i = CT_i - AT_i$$
$$WT_i = TAT_i - BT_i$$
$$RT_i = ST_i - AT_i$$
$$AVG(WT) = (1/n) * sum(WT_i)$$

روابط پایه محاسبه شاخص‌ها

تفسیر شاخص‌ها

- زمان انتظار پایین به معنای تلف‌نشدن عمر پردازه در صف است.
- زمان چرخش برای کارهای دسته‌ای و زمان پاسخ برای کارهای تعاملی معیار اصلی است.
- پردازنده نسبت زمان اشغال به کل زمان است و در این مدل برابر ۱۰۰٪ است چون زمان‌بند کارگریز نیست. بهره‌وری
- تعداد تعویض زمینه، نماینده سربار زمان‌بندی است و باید همراه با سایر شاخص‌ها تفسیر شود.

مبانی نظری (۴): صف آماده، تعویض زمینه و بهره‌وری



شکل 4. نمایش تصویری Turnaround، Waiting، Burst و Response برای یک پردازه

تعویض زمینه (Context Switch)

- تعویض زمینه فرایند ذخیره وضعیت پردازه فعلی و بارگذاری وضعیت پردازه بعدی است.
- این عمل توسط سخت‌افزار انجام نمی‌شود و زمان آن هزینه‌ای مستقیم بر کارایی است.
- هزینه شامل ذخیره و بازیابی ثبات‌ها، به‌روزرسانی جدول صفحات و از دست رفتن محتوای حافظه نهان است.
- در این مدل، تعداد تعویض‌ها از ردیابی اجرا استخراج می‌شود تا با نمودار گانت سازگار باشد.

بهره‌وری و توان عملیاتی

```
CPU Utilization = Busy Time / Total Time
     Throughput = n / Total Time
```

بهره‌وری پردازنده و توان عملیاتی

- در تمام سناریوهای این تمرین، چون همیشه پردازه آماده وجود دارد، بهره‌وری ۱۰۰٪ است.
- بهره‌وری ۱۰۰٪ نشانه کیفیت الگوریتم نیست؛ بلکه نشانه کارگریز بودن زمان‌بند است.
- تفاوت الگوریتم‌ها در این سناریوها در زمان انتظار، زمان چرخش و زمان پاسخ دیده می‌شود.

معرفی MLQ: فلسفه و اهداف طراحی

تعریف

موردنیاز به یک صف تخصیص می‌یابند و پس از آن به صف دیگری منتقل نمی‌شوند (در مدل کلاسیک و بدون پیری). اولویت و سیاست زمان‌بندی خاص خود است. پردازه‌ها بر پایه ویژگی‌هایی مانند نوع کار، میزان اهمیت یا حافظه زمان‌بندی چندسطحی صف‌ها، مجموعه پردازه‌های آماده را به چند صف مجزا تقسیم می‌کند. هر صف دارای سطح

اهداف طراحی

- جداسازی سیاست: هر کلاس از کارها بهترین سیاست خود را دریافت می‌کند.
- کاهش سربار: نیازی نیست همه پردازه‌ها در یک ساختار واحد مرتب شوند.
- کنترل اولویت: می‌توان تضمین کرد کارهای بحرانی زودتر اجرا شوند.
- سادگی تحلیل: رفتار هر سطح را می‌توان مستقل بررسی کرد.
- پایداری: تغییر سیاست یک صف، سایر صف‌ها را مختل نمی‌کند.

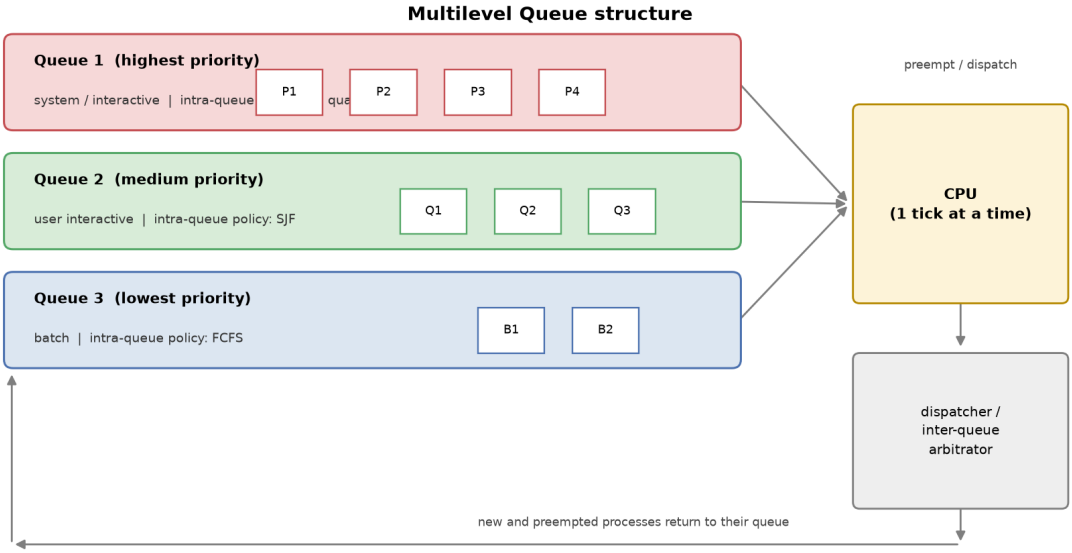
تفاوت با Multilevel Feedback Queue

MLFQ	MLQ	ویژگی
ثابت	ثابت	تعداد صف‌ها
دارد بر اساس رفتار	ندارد (در مدل کلاسیک)	جابه‌جایی بین صف‌ها
زیاد	کم	نیاز به پارامتر اضافی
بالا	پایین	پیچیدگی پیاده‌سازی

جدول 5. تمایز صف چندسطحی ساده و صف چندسطحی بازخوردی

رویکرد این گزارش: اختیاری برای جلوگیری از گرسنگی ارائه شده و در هر جدول نتایج، مقدار آن صریحاً ذکر می‌شود. در این تمرین، «پیری» تنها به‌عنوان یک افزونه

ساختار صف‌ها و نگاشت به کلاس‌های کاری



شکل 6. ساختار کلی صف چندسطحی شامل سه صف، سیاست‌های درون‌صفی و نگهبان بین‌صفی

اجزای ساختار

- مجموعه صف‌ها: در این پروژه سه صف با شماره ۱ تا ۳؛ شماره کوچک‌تر یعنی اولویت بالاتر.
- سیاست درون‌صفی: برای هر صف یکی از FCFS، RR، SJF یا SRTF.
- سیاست بین‌صفی: اولویت ثابت یا برش زمانی که در این نمودار «نگهبان» نامیده شده است.
- واحد اجرا: یک پردازنده که در هر تیک به یک پردازه تخصیص می‌یابد.

نگاشت پیشنهادی کلاس‌ها

دلیل	سیاست پیشنهادی	کلاس کاری	صف
پاسخ‌دهی سریع	با کوانتوم کوتاه RR	سیستمی و تعاملی بحرانی	صف ۱
کاهش میانگین انتظار	SRTF یا SJF	تعاملی عادی کاربر	صف ۲
سادگی و کمینه سربار	FCFS	دسته‌ای و پس‌زمینه	صف ۳

جدول 7. یک نگاشت عملی از کلاس‌های کاری به صف‌ها

تخصیص پردازه‌ها به صف‌ها

معیارهای تخصیص

- نوع پردازه: سیستمی، تعاملی، دسته‌ای یا بلادرنگ.
- حافظه موردنیاز: پردازه‌های بزرگ به صف‌هایی با سیاست متفاوت می‌روند.
- اولویت اداری: اولویت تعیین‌شده توسط مدیر سیستم یا سیاست سازمان.
- ویژگی‌های اندازه‌گیری‌شده: میزان استفاده از I/O در اجراهای گذشته.
- حساسیت به زمان: کارهای دارای مهلت به صف بالاتر می‌روند.

قاعده تخصیص در این پیاده‌سازی

سناریو انجام می‌شود و الگوریتم صرفاً اعتبارسنجی می‌کند که شماره صف در محدوده صف‌های تعریف‌شده باشد. هر پردازه، مقدار page به‌صورت صریح تعیین می‌شود. این رویکرد شفاف‌ترین حالت است: تخصیص توسط طراح در ورودی JSON

خطاهای رایج در تخصیص

رفتار	پیام برنامه	خطا
خروج با کد ۲	queue 7 not in [1]	شماره صف نامعتبر
خروج با کد ۲	missing field 'burst'	فیلد گم‌شده
خروج با کد ۲	burst must be > 0	صفر یا منفی Burst
خروج با کد ۲	arrival must be >= 0	زمان ورود منفی

جدول 8. نمونه رفتار برنامه در برابر ورودی نامعتبر (خروجی واقعی)

نکته طراحی: با MLFQ است. اگر نیاز به جابه‌جایی بر اساس رفتار باشد، باید از MLFQ یا از افزونه پیری استفاده کرد. تخصیص ثابت، مهم‌ترین تفاوت MLQ

سیاست زمان‌بندی داخل هر صف

نخستین ورود، نخستین خروج — FCFS

- غیرپیش‌گیرانه و مبتنی بر ترتیب ورود.
- پیاده‌سازی با یک صف ساده و هزینه O(1) برای انتخاب.
- اثر Convoy: یک پردازه طولانی، بقیه را پشت سر خود معطل می‌کند.
- مناسب صف‌های دسته‌ای که سربار کم اهمیت دارد.

نوبت گردشی — RR

- پیش‌گیرانه با کوانتوم مشخص؛ پس از اتمام سهمیه، پردازه به ته صف می‌رود.
- زمان پاسخ خوبی برای کار تعاملی فراهم می‌کند.
- کوانتوم بسیار کوچک سربار تعویض زمینه را افزایش می‌دهد.
- کوانتوم بسیار بزرگ رفتار را به FCFS نزدیک می‌کند.

کوتاه‌ترین کار ابتدا — SJF

- غیرپیش‌گیرانه و از نظر تئوری بهینه برای میانگین زمان انتظار.
- نیازمند دانستن یا تخمین طول Burst است.
- خطر گرسنگی پردازه‌های طولانی در صورت ورود مداوم کارهای کوتاه.

کوتاه‌ترین زمان باقی‌مانده ابتدا — SRTF

- نسخه پیش‌گیرانه SJF؛ با ورود کار کوتاه‌تر، پردازه جاری متوقف می‌شود.
- میانگین زمان انتظار را نسبت به SJF باز هم کاهش می‌دهد.
- سربار بیشتری دارد و نیازمند محاسبه مداوم زمان باقی‌مانده است.

مدیریت اولویت بین صف‌ها

روش اول: اولویت ثابت (Fixed Priority)

ساده، قطعی و پیش‌بینی‌پذیر است، اما در صورت تراکم کار در صف‌های بالا باعث گرسنگی صف‌های پایین می‌شود. پردازه‌ای در صف بالاتر آماده شود، پردازه در حال اجرا از صف پایین‌تر بلافاصله پیش‌گیرانه متوقف می‌شود. این روش بالاترین صف غیرخالی همیشه برنده است. اگر

روش دوم: برش زمانی (Time Slice)

از نظر تئوری از گرسنگی جلوگیری می‌کند و زمان پاسخ را متوازن می‌سازد، اما سربار تعویض زمینه را افزایش می‌دهد. هر صف سهمیه‌ای از زمان دریافت می‌کند و پس از پایان سهمیه، نوبت به صف بعدی می‌رسد. این روش

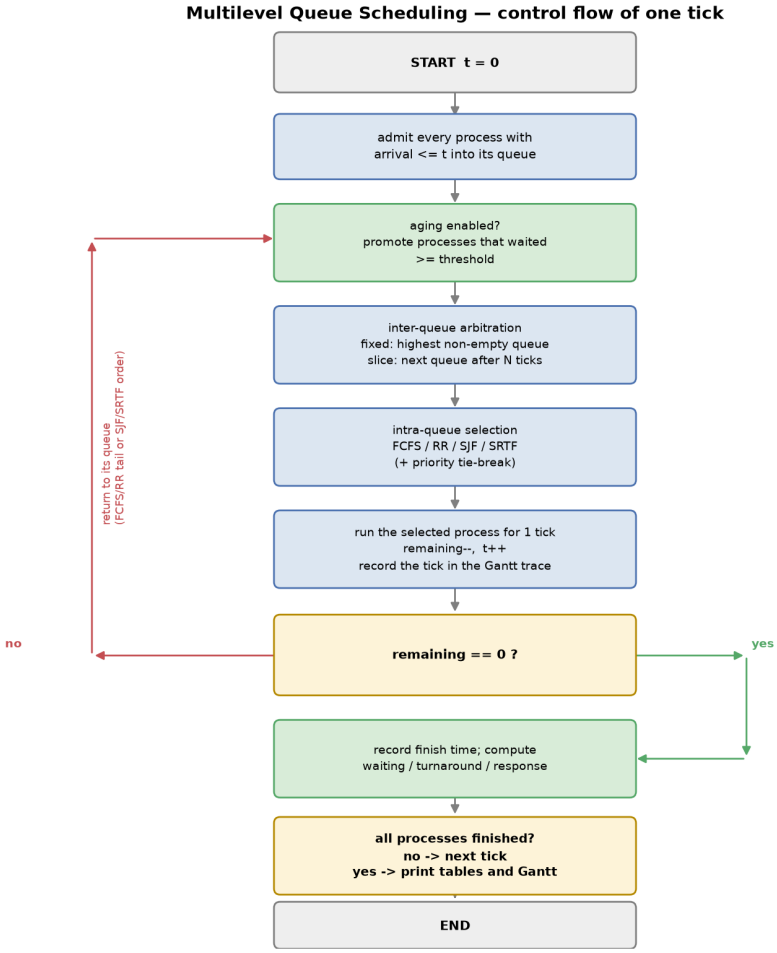
مقایسه عددی روی سناریوی timeslice

تعویض زمینه	میانگین پاسخ	میانگین چرخش	میانگین انتظار	سیاست بین‌صفی
۵	۷٫۸۰	۱۳٫۲۰	۸٫۴۰	اولویت ثابت
۱۲	۳٫۶۰	۱۹٫۰۰	۱۴٫۲۰	برش زمانی = ۲
۸	۶٫۰۰	۱۷٫۶۰	۱۲٫۸۰	برش زمانی = ۳
۶	۸٫۶۰	۱۸٫۰۰	۱۳٫۲۰	برش زمانی = ۵

جدول 9. نتایج واقعی اجرای سناریوی timeslice.json با چهار پیکربندی

- برش زمانی با سهمیه ۲، زمان پاسخ را از ۷٫۸۰ به ۳٫۶۰ کاهش داد (بهبود حدود ۵۴٪).
- همین تغییر، میانگین زمان انتظار را از ۸٫۴۰ به ۱۴٫۲۰ افزایش داد.
- تعداد تعویض زمینه از ۵ به ۱۲ رسید؛ یعنی افزایش سربار.
- انتخاب بین این دو روش یک بده‌بستان صریح بین پاسخ‌دهی و سربار است.

فلوچارت الگوریتم



شکل 10. فلوچارت کامل یک تیک از اجرای زمان‌بند چندسطحی

خواندن فلوچارت

- گام ۱: پذیرش همه پردازه‌هایی که زمان ورود آن‌ها فرا رسیده است.
- گام ۲: در صورت فعال‌بودن پیری، ارتقای پردازه‌های معطل‌مانده.
- گام ۳: نگهبان بین‌صفی تصمیم می‌گیرد کدام صف حق استفاده از پردازنده را دارد.
- گام ۴: سیاست درون‌صفی، پردازه منتخب را تعیین می‌کند.
- گام ۵: یک تیک اجرا، کاهش زمان باقی‌مانده و ثبت در ردیابی.
- گام ۶: اگر پردازه تمام شده، زمان پایان ثبت می‌شود؛ در غیر این صورت به صف بازمی‌گردد.

شبه‌کد (بخش ۱): حلقه اصلی زمان‌بند

```
MLQ_SCHEDULER(processes, queue_config, aging, inter_policy, inter_slice):
    levels  <- sorted(queue_config.keys())        # 1 = highest priority
    queues  <- { q: empty FIFO list  for q in levels }
    pending <- sort(processes by (arrival, pid))
    t <- 0 ;  current <- NULL ;  slice_used <- 0 ;  budget <- inter_slice
    while unfinished processes remain:
        ADMIT(t)                                  # arrival <= t joins its queue
        if aging > 0:  APPLY_AGING(t, aging)      # promote long waiters
        top <- the lowest-numbered non-empty queue, or NULL

        # ---- inter-queue arbitration ----
        if inter_policy == FIXED:
            if current != NULL and top < current.queue:
                push_front(queues[current.queue], current) ; current <- NULL
        else:  # TIME SLICE
            if current != NULL and budget <= 0:
                push_back(queues[current.queue], current)  ; current <- NULL
                budget <- inter_slice

        # ---- intra-queue selection ----
        if current == NULL and top != NULL:
            q    <- SELECT_QUEUE(top, levels, inter_policy)
            current <- SELECT_PROCESS(queues[q], queue_config[q].policy)
            remove current from queues[q]
            if current.first_start == NULL: current.first_start <- t
            slice_used <- 0
```

شبه‌کد بخش اول: پذیرش، داوری بین‌صفی و انتخاب پردازه

- SELECT_QUEUE در حالت اولویت ثابت، بالاترین صف غیرخالی را برمی‌گرداند.
- SELECT_QUEUE در حالت برش زمانی، صف بعدی را در یک چرخه دوری انتخاب می‌کند.
- SELECT_PROCESS بر اساس سیاست صف، یکی از FCFS، RR، SJF یا SRTF را اعمال می‌کند.

شبه‌کد (بخش ۲): اجرا، پیش‌گیری و پیری

```
        # ---- SRTF intra-queue preemption ----
        if current != NULL and queue_config[current.queue].policy == SRTF:
            best <- the ready process in current.queue with min remaining
            if best.remaining < current.remaining:  swap(current, best)

        if current == NULL:
            trace.append(IDLE) ; t <- t + 1 ; continue

        trace.append(current.pid)
        current.remaining <- current.remaining - 1
        slice_used <- slice_used + 1 ;  budget <- budget - 1 ;  t <- t + 1

        if current.remaining == 0:
            current.finish <- t ;  current <- NULL ;  budget <- 0
        else if (policy == RR and slice_used >= quantum)
                or (inter_policy == SLICE and budget <= 0):
            ADMIT(t)                              # new arrivals go first
            push_back(queues[current.queue], current) ; current <- NULL

    # ---- metrics ----
    for p in processes:
        p.turnaround = p.finish - p.arrival
        p.waiting    = p.turnaround - p.burst
        p.response   = p.first_start - p.arrival
    return trace, metrics, per_queue_statistics

APPLY_AGING(t, threshold):
    for each queue q from the second-highest downwards:
        for p in queues[q]:
            if t - p.wait_since >= threshold:
                move p one level up ; p.wait_since <- t ; promotions <- promotions + 1
```

شبه‌کد بخش دوم: اجرای یک تیک، شرایط پیش‌گیری و سازوکار پیری

نکات تحلیلی شبه‌کد

- هر تیک دقیقاً یک واحد از Burst پردازه منتخب را مصرف می‌کند.
- پیش‌گیری در سه نقطه رخ می‌دهد: صف بالاتر، پایان کوانتوم، و SRTF.
- پیری تنها یک سطح ارتقا می‌دهد تا رفتار الگوریتم قابل‌کنترل بماند.

مثال عددی حل‌شده (۱): صورت مسئله

صورت مسئله

پنج پردازه در زمان‌های مختلف وارد می‌شوند. مطلوب است تعیین ترتیب اجرا، نمودار گانت و شاخص‌های زمان‌بندی. سه صف با سیاست‌های زیر داریم: صف ۱ با RR و کوانتوم ۲، صف ۲ با SJF، و صف ۳ با FCFS.

سیاست صف	صف	Burst	زمان ورود	پردازه
FCFS	3	5	0	P1
RR (q=2)	1	4	1	P2
SJF	2	3	2	P3
RR (q=2)	1	2	3	P4
SJF	2	1	4	P5

جدول 11. داده‌های مثال عددی اول (سناریوی baseline.json)

قواعد اجرا در این مثال

- داوری بین‌صفی با اولویت ثابت است: صف ۱ بر صف ۲ و صف ۲ بر صف ۳ مقدم است.
- پیش‌گیری مجاز است؛ ورود یک پردازه به صف بالاتر پردازه جاری را متوقف می‌کند.
- در صف ۱ پس از دو تیک، پردازه به ته صف بازمی‌گردد.
- در صف ۲، کوتاه‌ترین Burst انتخاب می‌شود و پس از انتخاب، تا پایان اجرا ادامه می‌یابد.
- پیری غیرفعال است تا رفتار خالص الگوریتم دیده شود.

مثال عددی حل‌شده (۲): اجرای گام‌به‌گام

توضیح	تصمیم	بازه زمانی
پردازه آماده است و از صف ۳ اجرا می‌شود تنها	P1	۰ تا ۱
پیش‌گیری P1 می‌شود؛ دو تیک سهمیه RR ورود به صف ۱ باعث	P2	۱ تا ۳
Burst در صف ۱ وارد شده و تا پایان P4 اجرا می‌شود	P4	۳ تا ۵
ادامه دو تیک باقی‌مانده P2	P2	۵ تا ۷
SJF کوتاه‌ترین کار را انتخاب می‌کند صف ۲ با سیاست	P5	۷ تا ۸
تنها پردازه باقی‌مانده در صف ۲	P3	۸ تا ۱۱
به صف ۳ و اتمام چهار تیک باقی‌مانده بازگشت	P1	۱۱ تا ۱۵

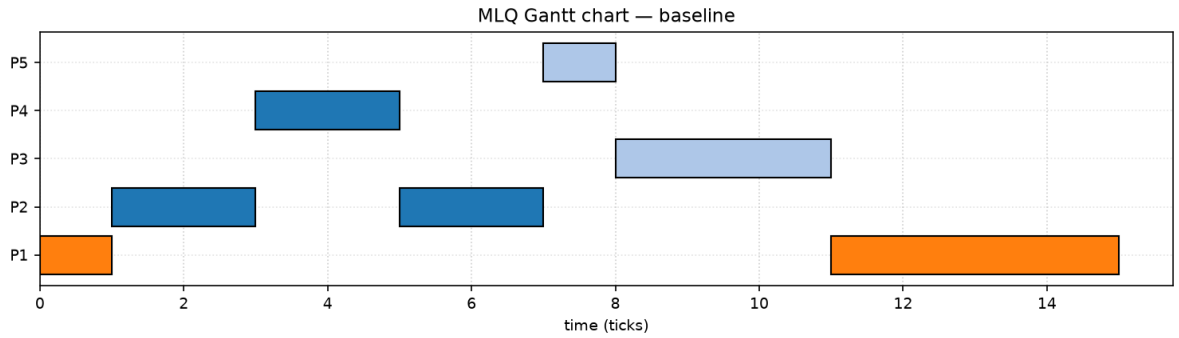
جدول 12. ردیابی گام‌به‌گام مثال عددی اول

بررسی تصمیم‌های کلیدی

- در t=1، P2 به صف ۱ می‌رسد و چون اولویت ثابت است، P1 از صف ۳ متوقف می‌شود.
- در t=3، سهمیه RR پردازه P2 تمام شده و P4 که تازه وارد شده در صف ۱ جلو می‌افتد.
- در t=7، صف ۲ دارای P3 با Burst 3 و P5 با Burst 1 است؛ SJF پردازه P5 را برمی‌گزیند.
- در t=11، صف‌های ۱ و ۲ خالی شده‌اند و P1 چهار تیک باقی‌مانده خود را اجرا می‌کند.

تطبیق با خروجی برنامه: ردیابی همان ردیابی تولیدشده توسط برنامه است و در فایل output/scenario_baseline.txt ذخیره شده است.

مثال عددی حل‌شده (۳): نمودار گانت



شکل 13. نمودار گانت مثال عددی اول، تولیدشده از اجرای واقعی برنامه

اعتبارسنجی نمودار

- مجموع طول بازه‌های غیر idle برابر ۱۵ و مجموع Burstها برابر ۵+۴+۳+۲+۱=۱۵ است.
- هیچ بازه idle وجود ندارد؛ بنابراین بهره‌وری پردازنده برابر ۱۰۰٪ است.
- تعداد بازه‌ها برابر ۷ است که با ۷ نوبت تخصیص پردازنده مطابقت دارد.
- تعداد تعویض‌های پردازه‌به‌پردازه برابر ۶ و تعداد پیش‌گیری‌ها برابر ۲ است.

مثال عددی حل‌شده (۴): جدول محاسبات و نتایج

پردازه	صف	ورود	Burst	پایان	انتظار	چرخش	پاسخ
P1	3	0	5	15	10	15	0
P2	1	1	4	7	2	6	0
P3	2	2	3	11	6	9	6
P4	1	3	2	5	0	2	0
P5	2	4	1	8	3	4	3

جدول 14. نتایج کامل مثال عددی اول (ارقام با خروجی برنامه یکسان است)

نمونه محاسبه دستی

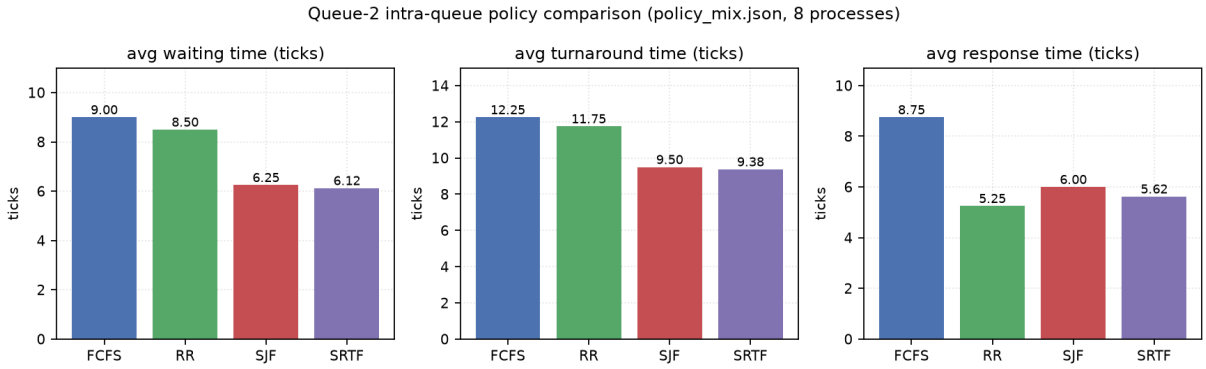
- P1: چرخش = ۱۵ − ۰ = ۱۵؛ انتظار = ۱۵ − ۵ = ۱۰؛ پاسخ = ۰ − ۰ = ۰.
- P2: چرخش = ۷ − ۱ = ۶؛ انتظار = ۶ − ۴ = ۲؛ پاسخ = ۱ − ۱ = ۰.
- P3: چرخش = ۱۱ − ۲ = ۹؛ انتظار = ۹ − ۳ = ۶؛ پاسخ = ۸ − ۲ = ۶.
- P4: چرخش = ۵ − ۳ = ۲؛ انتظار = ۲ − ۲ = ۰؛ پاسخ = ۳ − ۳ = ۰.
- P5: چرخش = ۸ − ۴ = ۴؛ انتظار = ۴ − ۱ = ۳؛ پاسخ = ۷ − ۴ = ۳.
- میانگین انتظار = (۱۰+۲+۶+۰+۳)/۵ = ۴٫۲۰ و میانگین چرخش = (۱۵+۶+۹+۲+۴)/۵ = ۷٫۲۰.

آمار هر صف

صف	تعداد	میانگین انتظار	میانگین چرخش	بیشینه چرخش
1	2	1.00	4.00	6
2	2	4.50	6.50	9
3	1	10.00	15.00	15

جدول 15. توزیع شاخص‌ها به تفکیک صف در مثال اول

سناریوی دوم: مقایسه سیاست‌های درون‌صفی



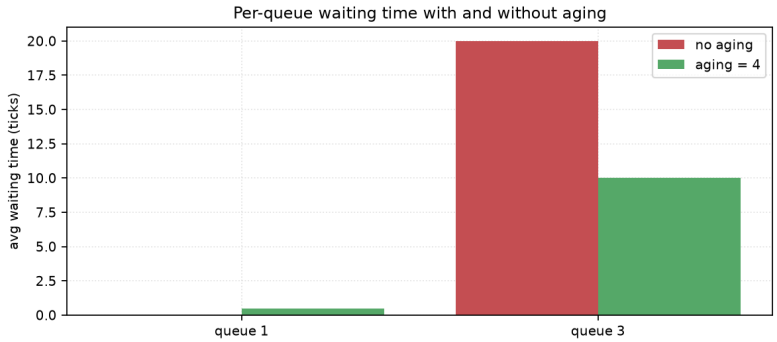
شکل 16. مقایسه چهار سیاست درون‌صفی روی یک بار کاری یکسان (policy_mix.json)

تعویض زمینه	میانگین پاسخ	میانگین چرخش	میانگین انتظار	سیاست صف ۲
8	8.75	12.25	9.00	FCFS
13	5.25	11.75	8.50	RR (q=2)
8	6.00	9.50	6.25	SJF
9	5.62	9.38	6.12	SRTF

جدول 17. نتایج واقعی مقایسه سیاست‌های درون‌صفی

- SRTF کمترین میانگین انتظار (۶٫۱۲) و کمترین میانگین چرخش (۹٫۳۸) را دارد.
- RR از ۸٫۷۵ به ۵٫۲۵ بهبود داد، اما تعویض زمینه را از ۸ به ۱۳ رساند. FCFS زمان پاسخ را نسبت به
- اختلاف SJF و SRTF در این بار کاری اندک است، زیرا پیش‌گیری فرصت کمی برای اثرگذاری دارد.
- این نتایج مختص همین بار کاری است و نباید به‌عنوان برتری مطلق تعمیم داده شود.

تحلیل گرسنگی در صف چندسطحی



شکل 18. زمان انتظار به تفکیک صف در سناریوی گرسنگی، با و بدون پیری

تعریف گرسنگی

اولویت ثابت، هرگاه جریان پیوسته‌ای از کار در صف‌های بالا وجود داشته باشد، صف‌های پایین هرگز نوبت نمی‌گیرند. پردازه آماده برای مدت نامحدود منتظر می‌ماند، زیرا منابع همواره به پردازه‌های دیگر تخصیص می‌یابد. در MLQ با گرسنگی وضعیتی است که در آن یک

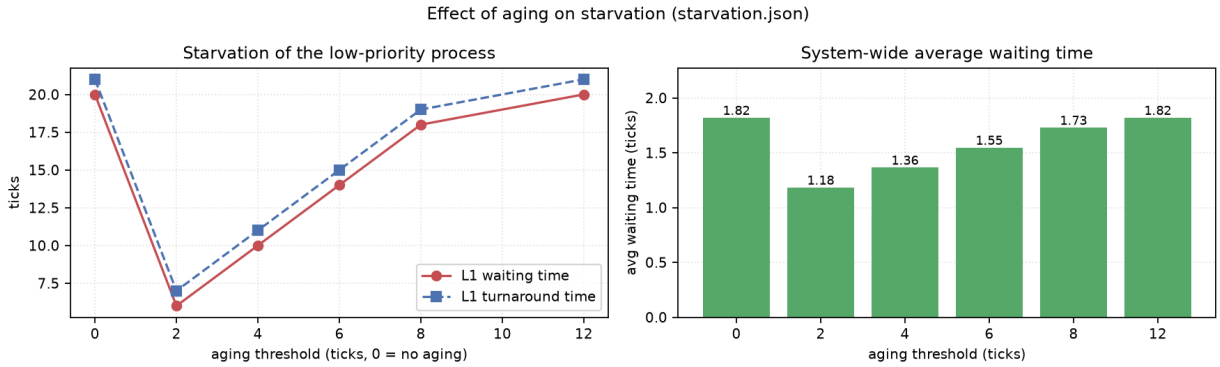
شواهد عددی

شاخص	صف ۱	صف ۳
تعداد پردازه	10	1
میانگین زمان انتظار	0.00	20.00
میانگین زمان چرخش	2.00	21.00
زمان پاسخ	0.00	20.00

جدول 19. سناریوی starvation.json بدون پیری: شکاف کامل بین صف بالا و پایین

- پردازه L1 در صف ۳ با Burst برابر ۱، باید ۲۰ تیک منتظر بماند.
- زمان انتظار با طول جریان صف بالا رابطه خطی دارد: به‌ازای هر پردازه اضافه، ۲ تیک افزوده می‌شود.
- این رفتار در تست خودکار با سه طول مختلف جریان (۲، ۵ و ۱۰) تأیید شده است.
- بهره‌وری در این سناریو همچنان ۱۰۰٪ است؛ بنابراین این شاخص گرسنگی را نشان نمی‌دهد.

روش‌های جلوگیری از گرسنگی



شکل 20. اثر آستانه پیری بر زمان انتظار پردازه کم‌اولویت و میانگین سیستم

سازوکار پیری

پردازه در نهایت به صف بالا می‌رسد و اجرا می‌شود. پیری در واقع اولویت را به تابعی از زمان انتظار تبدیل می‌کند. پردازه‌ای که بیش از یک آستانه مشخص در صف آماده منتظر مانده باشد، یک سطح ارتقا می‌یابد. با تکرار این فرایند، در پیری، هر

تعداد ارتقا	میانگین انتظار	چرخش L1	انتظار L1	آستانه پیری
0	1.82	21	20	0 (غیرفعال)
2	1.18	7	6	2
2	1.36	11	10	4
2	1.55	15	14	6
2	1.73	19	18	8
1	1.82	21	20	12

جدول 21. جاروب آستانه پیری روی سناریوی starvation.json (خروجی واقعی)

- با آستانه ۲، زمان انتظار L1 از ۲۰ به ۶ کاهش یافت؛ یعنی بهبود ۷۰ درصدی.
- هم‌زمان میانگین انتظار کل سیستم از ۱٫۸۲ به ۱٫۱۸ کاهش یافت، نه افزایش.
- آستانه بزرگ‌تر باعث دیرتر ارتقا یافتن پردازه می‌شود و اثر پیری را کم می‌کند.
- پیری رفتار MLQ کلاسیک را تغییر می‌دهد؛ لذا باید به‌صورت صریح در گزارش‌ها اعلام شود.

روش‌های دیگر

- برش زمانی بین صف‌ها: تضمین می‌کند هر صف سهمی از پردازنده بگیرد.
- سهمیه‌بندی اولویت: محدود کردن درصد زمان پردازنده برای هر سطح.
- ارتقای دستی: تغییر سطح توسط مدیر سیستم در موارد خاص.
- استفاده از MLFQ: جابه‌جایی خودکار بر اساس رفتار واقعی پردازه.

تحلیل پیچیدگی زمانی و فضایی

نمادگذاری

Burst برابر مجموع طول‌های T .طول کل افق شبیه‌سازی بر حسب تیک است T تعداد صف‌ها، و Q ،تعداد پردازه‌ها n .است idle به‌علاوه زمان‌های

پیچیدگی زمانی

توضیح	هزینه	عملیات
مرتب‌سازی اولیه بر اساس زمان ورود	O(n log n)	پذیرش پردازه‌ها
صف‌ها برای یافتن بالاترین صف غیرخالی بررسی خطی	O(Q)	انتخاب صف
برداشت از ابتدای صف	O(1)	انتخاب در FCFS/RR
کمینه روی m پردازه آماده همان صف	O(m)	انتخاب در SJF/SRTF
بسته به سیاست صف	O(m) تا O(1)	هر تیک اجرا
حاصل‌ضرب تعداد تیک و هزینه هر تیک	O(T · (Q + m))	کل شبیه‌سازی

جدول 22. خلاصه پیچیدگی زمانی اجزای شبیه‌ساز

پیچیدگی فضایی

- ذخیره پردازه‌ها و صف‌ها: O(n).
- نگاشت پیکربندی صف‌ها: O(Q).
- ردیابی اجرا برای نمودار گانت: O(T) در بدترین حالت.
- در این پیاده‌سازی، ردیابی به‌صورت فشرده ذخیره می‌شود و معمولاً بسیار کوچک‌تر از T است.

مقایسه با زمان‌بند واقعی

- زمان‌بند واقعی باید در زمان ثابت یا لگاریتمی تصمیم بگیرد؛ لذا از هرم یا صف اولویت‌دار استفاده می‌کند.
- در این مدل آموزشی، هدف شفافیت است و هزینه خطی قابل‌قبول است.
- برای n بزرگ می‌توان انتخاب را با صف اولویت‌دار به O(log m) کاهش داد.

پیاده‌سازی پایتون (۱): ساختار و چرخه استفاده

ساختار پوشه‌ها

```
exercise-1/
  README.md                راهنمای شروع سریع و معرفی فایل‌ها
  REPORT_FA.md             همین گزارش به صورت Markdown
  REPORT_EN.md             گزارش انگلیسی
  requirements.txt         وابستگی‌های پایتون
  code/
    mlq.py                 هسته شبیه‌ساز اصلی و خط فرمان
    run_experiments.py     اجرای همه سناریوها و دلتوم‌ها
    make_diagrams.py       رسم نمودارهای فمه‌وه‌می، راخاس و تراچ‌ولف
    scenarios/*.json       سناریوهای متعدد و متعامل
  tests/test_mlq.py        تست‌های پایه
  tests/test_mlq_extra.py  تست‌های افزونه‌های جدید
  diagrams/                نمودارهای دلتوم‌شده
  output/                  خروجی متنی، CSV و JSON
  docs/                    گزارش Word و PDF
```

ساختار پوشه‌های تمرین اول

اجرای سریع

```
cd exercise-1
python code/mlq.py code/scenarios/baseline.json
python code/mlq.py code/scenarios/policy_mix.json --policy 2=SRTF --quiet
python code/mlq.py code/scenarios/baseline.json --png g.png --csv r.csv --json r.json
python code/mlq.py --interactive
python code/run_experiments.py
python code/make_diagrams.py
python -m pytest tests/ -q
```

دستورهای اجرای پروژه (همگی آزمایش شده‌اند)

پیاده‌سازی پایتون (۲): ورودی، خروجی و مدیریت خطا

قالب ورودی

```
{
  "inter_policy": "fixed",        # fixed | slice
  "inter_slice": 2,
  "aging": 0,
  "queues": {
    "1": {"policy": "RR", "quantum": 2},
    "2": {"policy": "SJF"},
    "3": {"policy": "FCFS"}
  },
  "processes": [
    {"pid": "P1", "arrival": 0, "burst": 5, "queue": 3},
    {"pid": "P2", "arrival": 1, "burst": 4, "queue": 1, "priority": 0}
  ]
}
```

قالب فایل سناریو (نمونه واقعی baseline.json)

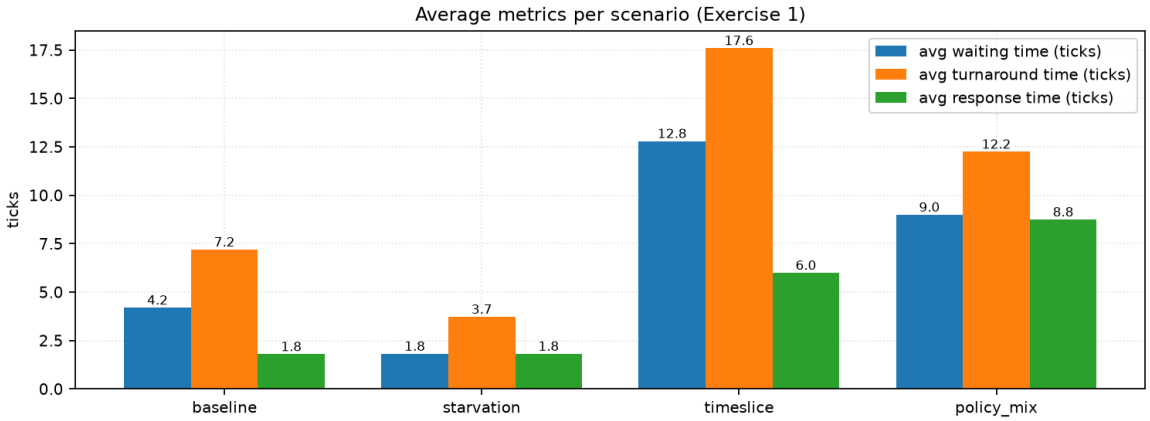
مدیریت خطا

- همه خطاهای ورودی از نوع InputError هستند و با کد خروجی ۲ پایان می‌یابند.
- بررسی‌ها شامل تهی‌بودن فهرست، تکراری‌بودن شناسه، منفی‌بودن زمان ورود و صفر یا منفی‌بودن Burst است.
- سیاست ناشناخته یا کوانتوم نامعتبر نیز پیش از اجرا شناسایی می‌شود.
- فایل‌های سناریوی نامعتبر در پوشه scenarios نگه داشته شده‌اند تا رفتار خطا قابل بازتولید باشد.
- یک نگهبان max_ticks از حلقه‌کردن بی‌پایان در صورت بروز اشکال جلوگیری می‌کند.

پوشش تستی

- ۵۹ تست خودکار با pytest اجرا شده و همه آن‌ها موفق بوده‌اند.
- تست‌ها شامل درستی گانت، اتحادهای شاخص‌ها، سیاست‌ها، گرسنگی، پیری و خطای ورودی هستند.
- تست‌های جدید تعداد تعویض زمینه را مستقیماً با نمودار گانت مقایسه می‌کنند.
- خروجی کامل اجرای تست‌ها در فایل output/pytest_result.txt ذخیره شده است.

سناریوها و نتایج آزمایش‌ها



شکل 23. میانگین شاخص‌ها در چهار سناریوی آزمایشی تمرین اول

سناریو	تعداد	میانگین انتظار	میانگین چرخش	میانگین پاسخ	بهره‌وری	تعویض زمینه
baseline	5	4.20	7.20	1.80	100%	6
starvation	11	1.82	3.73	1.82	100%	10
timeslice	5	12.80	17.60	6.00	100%	8
policy_mix	8	9.00	12.25	8.75	100%	8

جدول 24. خلاصه نتایج همه سناریوها (از output/experiments.json)

معرفی سناریوها

- baseline: سناریوی مرجع آموزشی؛ ترکیب سه صف با سه سیاست متفاوت و پنج پردازه.
- starvation: برای سنجش گرسنگی؛ ده پردازه در صف بالا و یک پردازه در صف پایین.
- timeslice: پنج پردازه با Burst زمانی برش ثابت و بزرگ برای مقایسه اولویت.
- policy_mix: هشت پردازه که پنج تای آن‌ها در صف ۲ هستند تا سیاست‌های درون‌صفی مقایسه شوند.

تحلیل نتایج

مشاهده‌های اصلی

- بهره‌وری در همه سناریوها ۱۰۰٪ است، زیرا زمان‌بند کارگریز است و هیچ idle ایجاد نمی‌شود.
- بیشترین میانگین انتظار مربوط به سناریوی timeslice (۱۲٫۸۰) است؛ نتیجه مستقیم تقسیم زمان بین صف‌ها.
- کمترین میانگین انتظار مربوط به سناریوی starvation (۱٫۸۲) است؛ اما این عدد گمراه‌کننده است، زیرا پردازه کم‌اولویت ۲۰ تیک منتظر مانده است.
- این تضاد نشان می‌دهد که میانگین‌ها به‌تنهایی برای قضاوت کافی نیستند و باید پراکندگی هم گزارش شود.

بده‌بستان‌های مشاهده‌شده

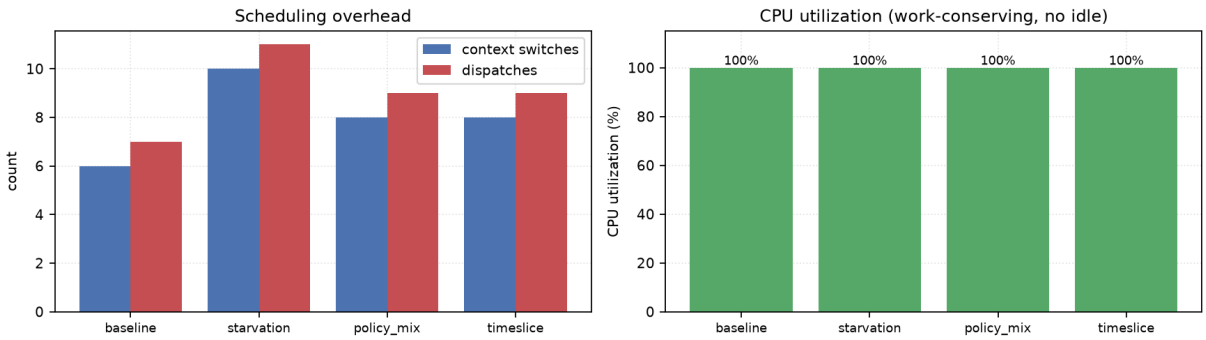
هزینه	سود	تصمیم طراحی
گرسنگی صف پایین	زمان پاسخ عالی برای صف بالا	اولویت ثابت
تعویض زمینه و انتظار بیشتر	پاسخ متوازن برای همه صف‌ها	برش زمانی
سربار بیشتر	پاسخ‌دهی بهتر	با کوانتوم کوتاه RR
نیاز به دانستن Burst و خطر گرسنگی	انتظار کمتر	SJF/SRTF
انحراف از مدل کلاسیک MLQ	رفع گرسنگی	پیری

جدول 25. بده‌بستان‌های اصلی در طراحی صف چندسطحی

اعتبارسنجی متقابل

- مثال عددی اول به‌صورت دستی محاسبه و با خروجی برنامه تطبیق داده شد؛ همه ارقام یکسان بودند.
- تست‌های خودکار اتحاد «انتظار = چرخش - Burst» را برای همه سناریوها بررسی می‌کنند.
- تعداد تعویض زمینه از ردیابی استخراج می‌شود، نه از شمارش جداگانه؛ بنابراین با گانت سازگار است.

سربار زمان‌بندی و هزینه تصمیم‌گیری



شکل 26. تعداد تعویض زمینه، تخصیص‌ها و بهره‌وری پردازنده در سناریوها

یافته‌ها

- سناریوی starvation با ۱۱ پردازه، ۱۰ تعویض زمینه دارد؛ یعنی تقریباً یک تعویض به‌ازای هر پردازه.
- سناریوی baseline با ۵ پردازه و ۷ تخصیص، ۶ تعویض زمینه دارد که ناشی از پیش‌گیری‌های صف ۱ است.
- در سیاست برش زمانی با سهمیه ۲، تعداد تعویض‌ها به ۱۲ رسید؛ افزایش ۱۴۰ درصدی نسبت به اولویت ثابت.
- بهره‌وری در این مدل تحت‌تأثیر تعویض زمینه قرار نمی‌گیرد، زیرا هزینه تعویض صفر فرض شده است.

محدودیت اندازه‌گیری: حافظه نهان را باطل می‌کند؛ بنابراین این نمودار باید به‌عنوان کران پایین سربار تفسیر شود. در سیستم واقعی، هر تعویض زمینه چند میکروثانیه هزینه دارد و محتوای

مزایا، معایب و مقایسه با سیاست‌های تک‌صفی

مزایا

- جداسازی سیاست‌ها و امکان بهینه‌سازی مستقل هر کلاس کاری.
- کنترل صریح اولویت و تضمین پاسخ‌دهی برای کارهای بحرانی.
- سربار کم در صف‌هایی که از FCFS استفاده می‌کنند.
- سادگی پیاده‌سازی نسبت به الگوریتم‌های بازخوردی.
- قابلیت ترکیب با سیاست‌های پیشرفته در هر سطح.

معایب

- گرسنگی صف‌های پایین در صورت تراکم کار در صف‌های بالا.
- عدم انعطاف: پردازه پس از تخصیص، در مدل کلاسیک جابه‌جا نمی‌شود.
- نیاز به تنظیم دستی پارامترها: تعداد صف‌ها، کوانتوم و سیاست هر صف.
- وابستگی شدت نتایج به کیفیت تخصیص اولیه پردازه به صف.
- پایین اگر صف بالا همیشه اشغال و صف پایین همیشه آماده باشد و سیاست بین‌صفی محدودکننده باشد. احتمال بهره‌وری

مقایسه با سیاست‌های تک‌صفی

MLQ	SJF	RR	FCFS	معیار
خوب (صف بالا)	متوسط	خوب	ضعیف	پاسخ‌دهی تعاملی
وابسته به تخصیص	بهینه	متوسط	بالا	میانگین انتظار
کم تا متوسط	کم	متوسط	کم	سربار
دارد (صف پایین)	دارد	ندارد	ندارد	خطر گرسنگی
بالا	کم	کم	کم	انعطاف‌پذیری

جدول 27. مقایسه MLQ با سیاست‌های تک‌صفی

محدودیت‌ها، فرضیات و کاربردهای واقعی

فرضیات مدل

- زمان به‌صورت گسسته و بر حسب تیک مدل شده است.
- طول Burst از پیش معلوم فرض می‌شود؛ در سیستم واقعی باید تخمین زده شود.
- هزینه تعویض زمینه صفر در نظر گرفته شده و بر شاخص‌ها اثر نمی‌گذارد.
- یک پردازنده واحد وجود دارد؛ مسئله چندپردازنده‌ای بررسی نشده است.
- پردازه‌ها مستقل و بدون وابستگی و بدون نیاز به همگام‌سازی هستند.
- هیچ عملیات ورودی/خروجی در میانه Burst رخ نمی‌دهد.

محدودیت‌های آزمایش

- سناریوها مصنوعی و کوچک‌اند تا محاسبات دستی ممکن باشد.
- نتایج به همین سناریوها وابسته است و قابل‌تعمیم خودکار به بارهای واقعی نیست.
- شبیه‌ساز در فضای کاربر اجرا می‌شود و رفتار زمان‌بند واقعی هسته را اندازه‌گیری نمی‌کند.
- شاخص‌هایی مانند انرژی مصرفی، اثر حافظه نهان و تأخیر حافظه اندازه‌گیری نشده‌اند.

کاربردهای واقعی

- جداسازی کارهای سیستمی از کاربری در سیستم‌عامل‌های اشتراک‌زمانی.
- اولویت‌بندی تراکنش‌های بانکی یا پردازش‌های پرداخت نسبت به گزارش‌گیری.
- مدیریت صف‌های چاپ و صف‌های کار در سیستم‌های دسته‌ای.
- زمان‌بندی بسته‌ها در شبکه با کلاس‌های سرویس متفاوت.
- تخصیص منابع در پایگاه‌داده‌ها بین پرس‌وجوهای تعاملی و دسته‌ای.

نتیجه‌گیری و پیشنهادها

نتیجه‌گیری

مقابل، همین تصمیم میانگین زمان انتظار را از ۸٫۴۰ به ۱۴٫۲۰ افزایش داد و تعداد تعویض زمینه را از ۵ به ۱۲ رساند. تعاملی را به‌طور چشمگیری بهبود دهد؛ در سناریوی timeslice، زمان پاسخ از ۷٫۸۰ به ۳٫۶۰ واحد کاهش یافت. در داده شد که این الگوریتم با جداسازی صف‌ها و تعیین سیاست مستقل برای هر سطح، می‌تواند پاسخ‌دهی کارهای الگوریتم صف چندسطحی راهکاری ساختاریافته برای مواجهه با ناهمگونی بار کاری است. در این تمرین نشان

نیز بهبود یافت. این نتیجه نشان می‌دهد که هزینه رفع گرسنگی در این سناریو بسیار کمتر از فایده آن بوده است. با آستانه ۲، زمان انتظار پردازه کم‌اولویت را از ۲۰ واحد به ۶ واحد کاهش داد، در حالی که میانگین انتظار کل سیستم داده شد که در صورت نبود سازوکار محافظ، پردازه‌های صف‌های پایین دچار گرسنگی کامل می‌شوند. سازوکار پیری همچنین نشان

پیشنهادها برای ادامه کار

- افزودن صف بازخوردی (MLFQ) و مقایسه آن با MLQ روی یک بار کاری یکسان.
- مدل‌سازی هزینه واقعی تعویض زمینه و اثر آن بر بهره‌وری.
- افزودن عملیات ورودی/خروجی و اندازه‌گیری زمان پاسخ در حضور I/O.
- تعمیم مدل به چند پردازنده و بررسی توازن بار.
- استفاده از بارهای کاری واقعی به‌جای سناریوهای مصنوعی.

جمع‌بندی در یک جمله

پیام کلیدی: بیش از هر چیز به درستی تخصیص پردازه‌ها به صف‌ها و به تنظیم سیاست بین‌صفی وابسته است MLQ ابزاری قدرتمند اما پارامتر-محور است: کیفیت خروجی آن.

منابع

منابع اصلی

- Silberschatz, A., Galvin, P. B., & Gagne, G. — Operating System Concepts — فصل‌های مربوط به CPU Scheduling.
- Tanenbaum, A. S., & Bos, H. — Modern Operating Systems — بخش Scheduling.
- Stallings, W. — Operating Systems: Internals and Design Principles — فصل Scheduling.
- Liu, C. L., & Layland, J. W. — Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment — Journal of the ACM, 1973.

منابع تکمیلی

- مستندات رسمی زبان پایتون برای پودمان‌های collections و dataclasses.
- مستندات کتابخانه matplotlib برای تولید نمودارها.
- مستندات pytest برای نوشتن تست‌های خودکار.
- مستندات python-docx و reportlab برای تولید فایل Word و PDF.

وضعیت ارجاع‌ها: یا سال نشر هر ویراست، لازم است نسخه در دسترس دانشجو بررسی و ارجاع به‌روزرسانی شود. مراجع استاندارد و شناخته‌شده درس سیستم‌عامل فهرست شده‌اند. در صورت نیاز استاد به ذکر دقیق شماره صفحه منابع کتابی بالا بر اساس

پیوست: فهرست خروجی‌ها و وضعیت فایل‌ها

فایل‌های خروجی تولیدشده

توضیح	فایل
گانت و جدول کامل سناریوی مرجع	output/scenario_baseline.txt
نتایج سناریوی گرسنگی	output/scenario_starvation.txt
نتایج سناریوی برش زمانی	output/scenario_timeslice.txt
نتایج سناریوی مقایسه سیاست‌ها	output/scenario_policy_mix.txt
همه اعداد به صورت ماشین‌خوان	output/experiments.json
خلاصه متنی همه آزمایش‌ها	output/experiments_summary.txt
خروجی اجرای تست‌ها	output/pytest_result.txt
نمودار ساختار صف‌ها	diagrams/fig_mlq_structure.png
فلوچارت الگوریتم	diagrams/fig_flowchart.png
نمودارهای گانت هر سناریو	diagrams/gantt_*.png
نمودارهای مقایسه‌ای	diagrams/chart_*.png

جدول 28. فهرست فایل‌های خروجی تمرین اول

تطبیق با الزامات ارزیابی

محل ارائه	وضعیت	مورد ارزیابی
صفحات ۶ تا ۹	کامل	تعریف و مبانی نظری
صفحات ۱۸ تا ۲۱	کامل	مثال عددی حل‌شده
صفحه ۱۵	کامل	فلوچارت
صفحات ۱۶ و ۱۷	کامل	شبه‌کد
پوشه code و ۵۹ تست	کامل	کد اجرایی
صفحات ۱۴، ۲۲ و ۳۱	کامل	جدول مقایسه
REPORT_EN و REPORT_FA	کامل	نسخه فارسی و انگلیسی
پوشه docs	کامل	نسخه Word و PDF
مستند در فهرست اصلی پروژه	نامرتبط با این تمرین	کلیپ تمرین پنجم

جدول 29. وضعیت تطبیق تمرین اول با جدول ارزیابی (بدون پیش‌داوری درباره نمره)